



Lecture 09: Component Logic and the `<script>` Tag

INTERVIEW QUESTION

What is the role of the `<script>` tag in a Svelte component?

In a `.svelte` single-file component, the `<script>` block is the designated home for your component's state, data, and logic. It is where you import dependencies, declare variables, and define functions that your markup will interact with.

However, its most important characteristic is its execution model.

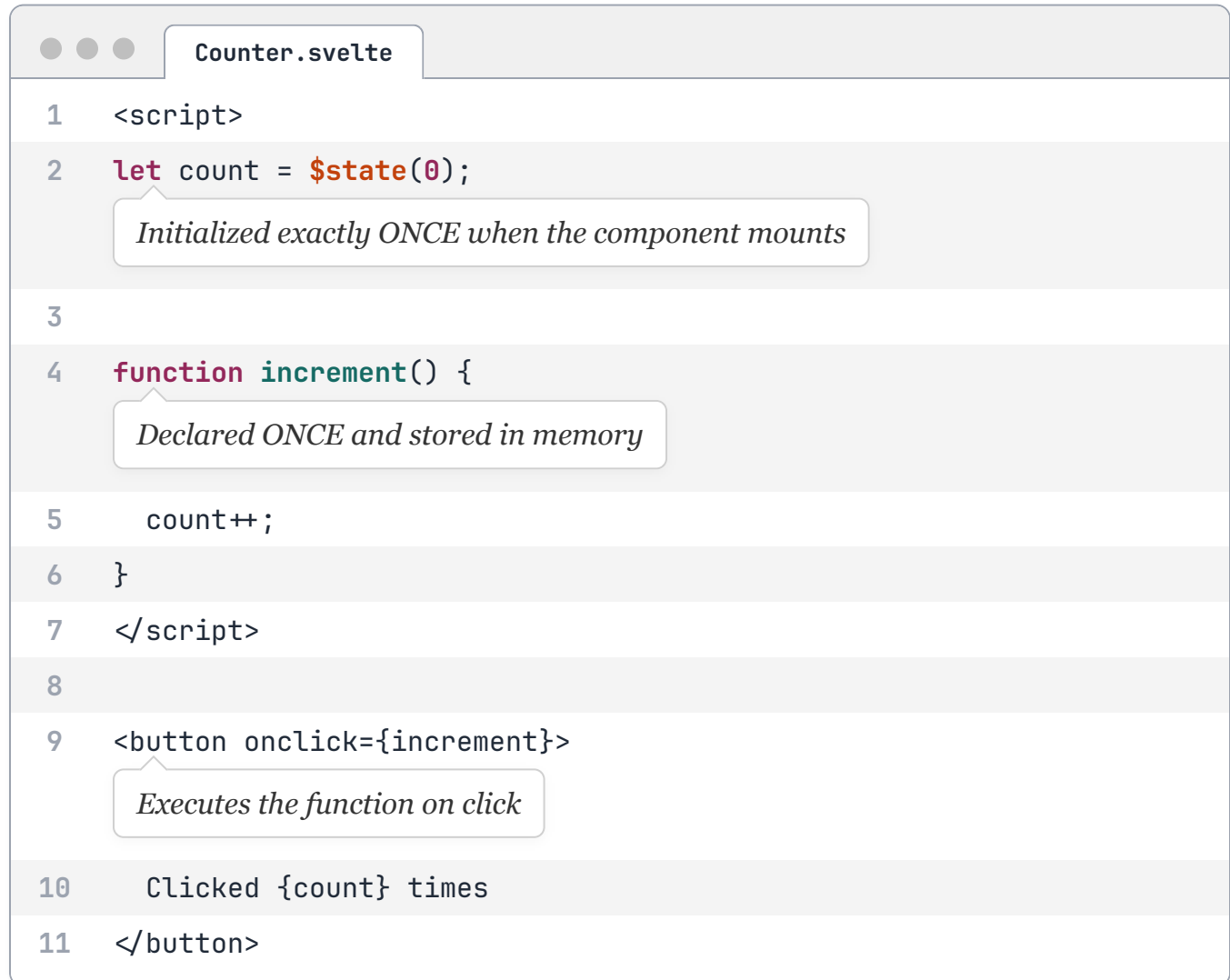
The Initialization Phase

Unlike frameworks like React, where the entire component function re-runs from top to bottom every time a piece of state changes, the code inside a Svelte `<script>` block executes **exactly once** when the component is instantiated.

It acts strictly as an initialization phase. When a component is mounted to the DOM, Svelte runs the `<script>` block to set up the initial state and wire up any event listeners. Once that setup is complete, the `<script>` block never executes again for the lifespan of that specific component instance.

A Worked Example

Consider a standard counter component. The variable declarations and function definitions happen only during creation.



```
1  <script>
2  let count = $state(0);
3
4  function increment() {
5    count++;
6  }
7  </script>
8
9  <button onclick={increment}>
10    Clicked {count} times
11  </button>
```

Callouts:

- Line 2: *Initialized exactly ONCE when the component mounts*
- Line 4: *Declared ONCE and stored in memory*
- Line 9: *Executes the function on click*

When the user clicks the button, `count` increases. In React, this state change would cause the entire component to re-execute to generate a new Virtual DOM tree. In Svelte, the `<script>` block does not re-run. Instead, Svelte's compiler has already wired the `count` signal directly to the text node in the `<button>`, so only that tiny piece of the DOM updates.

KEY TAKEAWAY

The `<script>` block is not a render function. It is a one-time initialization script that sets up the local state, imports, and logic for a specific component instance.

Isn't this common sense?

The idea that a component's setup script should only run once may seem like obvious common sense, but in the modern framework landscape, it is not.

Because React components are functions that re-execute on every state change, developers often accidentally trigger infinite render loops. A developer might write a simple `console.log()` or a data fetch, only to open their DevTools and see their app printing 100s of errors in rapid repetition—looking like the app is on fire. In Svelte, because the `<script>` runs exactly once, these endless re-evaluation bugs are architecturally impossible.

INTERVIEW TIP

To impress the interviewer: Explicitly highlight the performance implications of this execution model. Because the `<script>` block only runs once, you can perform heavy initial calculations or object allocations inside it without fear of those operations re-running on every keystroke or interaction. This eliminates the need for **memoization**—a technique where developers are forced to manually cache expensive calculations so the framework remembers them instead of wasting CPU cycles re-doing the work on every render. While React developers rely heavily on complex hooks like `useMemo` or `useCallback` to patch this performance leak, Svelte's execution model makes it a non-issue.

Summary

The `<script>` tag encapsulates the JavaScript logic for a Svelte component. Because it runs precisely once during component creation, it provides a highly efficient environment for declaring state and functions without the runtime overhead of continuous re-evaluation during state changes.

	REACT COMPONENT (Runtime Render Function)	SVELTE <code><script></code> (Initialization Block)
Execution Frequency	Runs repeatedly on every state change	Runs exactly ONCE upon instantiation
Re-allocation Risk	High (functions/ objects recreated every render)	Zero (declared once in memory)
Memoization Required?	Yes (heavy reliance on <code>useMemo</code> / <code>useCallback</code>)	No (inherently optimized by the COMPILER)